

Алгоритм обратного распространения ошибок

1. Начнем с аналогий из жизни

Представьте, что вы учитесь бросать мяч в корзину с закрытыми глазами.

Вы бросили → мяч улетел на 2 метра правее корзины

Ваш друг говорит: «Промазал на 2 метра вправо!» — это **ошибка**.

Что вы делаете? Вы **корректируете** бросок — в следующий раз кидаете чуть выше. Причём чем сильнее промазали, тем сильнее корректируете.

Обратное распространение ошибки — это ровно то же самое, только для нейронной сети.

Сеть «бросает мяч» (предсказание), видит, наконец, проделала, и подкручивает наши внутренние настройки (веса), чтобы в следующий раз попасть точнее.

2. Что такое нейронная сеть?

Искусственную нейронную сеть уместно воспринимать как простую **цепочку математических операций**:

Входные данные → [умножение на веса + сложение] → [функция активации] → Результат

Главная идея: мы хотим подобрать такие w_1 , w_2 , b , чтобы сеть дала разумные ответы.

3. Два этапа обучения нейросетей

ЭТАП 1: Прямое распространение (Forward Pass)

Данные идут ВПЕРЁД: вход → выход

Получаем предсказание и считаем ошибку

ЭТАП 2: Обратное распространение (Backward Pass)

Ошибка идёт НАЗАД: выход → вход

Вычисляем, как каждый вес повлиял на ошибку

Корректируем веса

4. Подробный пример

МОМ

Предположим, у нас есть **один нейрон** с одним входом:

Вход (x) = 2

Правильный ответ (y) = 1

Начальный вес (w) = 0.5

Смещение (b) = 0.1

Функция активации: сигмоида $\sigma(z) = 1 / (1 + e^{-z})$

Скорость обучения (α) = 0.1

$x=2 \xrightarrow{w=0.5} [z = x \cdot w + b] \xrightarrow{b=0.1} [\sigma(z)] \rightarrow \hat{y} \rightarrow \text{сравниваем с } y=1$

ЭТАП 1: Прямое распространение

Просто посмотрите по формулам слева направо :

Шаг 1.1: Вычисляем взвешенную сумму

$$z = x \cdot w + b$$

$$z = 2 \cdot 0.5 + 0.1$$

$$z = 1.1$$

Шаг 1.2: Примените функцию активации (сигмоиду)

$$\hat{y} = \sigma(z) = 1 / (1 + e^{-z})$$

$$\hat{y} = 1 / (1 + e^{-1.1})$$

$$\hat{y} = 1 / (1 + 0.3329)$$

$$\hat{y} = 1 / 1.3329$$

$$\hat{y} \approx 0.7503$$

Шаг 1.3: Считаю ошибку (функция потерь)

воздействие среднеквадратической ошибки (MSE):

$$\begin{aligned} L &= (y - \hat{y})^2 \\ L &= (1 - 0.7503)^2 \\ L &= (0.2497)^2 \\ L &\approx 0.0624 \end{aligned}$$

Итог прямого прохода: сеть предсказала 0.7503, а нужно 1. Ошибка = 0.0624.

ПРЯМОЙ ПРОХОД (слева направо):

$$\begin{aligned} x=2 &\longrightarrow z=1.1 \longrightarrow \hat{y}=0.7503 \longrightarrow L=0.0624 \\ w=0.5 &\quad b=0.1 \end{aligned}$$

ЭТАП 2: Обратное распространение (Обратный проход)

Теперь самое главное. Нам понять нужно:

«Как нужно изменить вес w и перемещение b , чтобы ошибка L уменьшилась?»

Для этого нам необходимо найти **производные** (градиенты):

- $\partial L / \partial w$ — наконец-то ошибка при повороте w
- $\partial L / \partial b$ — наконец-то ошибка при включении b

Зачем производные?

Представьте, что вы стоите на холме в тумане и хотите спуститься вниз. Вы щупаете землю вокруг себя и идёте туда, где наклон вниз самый крутой.

Производная (градиент) — это "наклон поверхности". Она показывает, в какую сторону и насколько сильно нужно изменить вес.



здесь хотим попасть

Цепное правило (Chain Rule) — ключ к обратному распространению ошибки

Наша сеть — это цепочка с функциями :

$$x \rightarrow [z = x \cdot w + b] \rightarrow [\hat{y} = \sigma(z)] \rightarrow [L = (y - \hat{y})^2]$$

Чтобы найти $\partial L / \partial w$, мы используем **цепное правило** (правило дифференцирования цифровых функций):

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Это как цепочка шестеренок:

Изменение $w \rightarrow$ влияет на $z \rightarrow$ влияет на $\hat{y} \rightarrow$ влияет на L

Считаем каждое звено цепочки **справа налево** (от выхода ко входу — поэтому «обратное» распространение!):

Звено 1: $\partial L / \partial \hat{y}$ (как ошибка зависит от предсказания)

$$L = (y - \hat{y})^2$$

$$\partial L / \partial \hat{y} = -2(y - \hat{y})$$

$$\partial L / \partial \hat{y} = -2(1 - 0.7503)$$

$$\partial L / \partial \hat{y} = -2 \cdot 0.2497$$

$$\partial L / \partial \hat{y} \approx -0.4994$$

Звено 2: $\partial \hat{y} / \partial z$ (как выход сигмоиды зависит от ее входа)

У сигмоидов есть замечательное свойство:

$$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z)) \quad \leftarrow \text{производная сигмоиды}$$

$$\partial \hat{y} / \partial z = \hat{y} \cdot (1 - \hat{y})$$

$$\partial \hat{y} / \partial z = 0.7503 \cdot (1 - 0.7503)$$

$$\partial \hat{y} / \partial z = 0.7503 \cdot 0.2497$$

$$\partial y / \partial z = 0.7503 \cdot 0.2497$$
$$\partial \hat{y} / \partial z \approx 0.1874$$

Звено 3: $\partial z / \partial w$ (как взвешенная длина зависит от веса)

$$z = x \cdot w + b$$

$$\partial z / \partial w = x = 2$$

Собираем цепочку:

$$\partial L / \partial w = (\partial L / \partial \hat{y}) \cdot (\partial \hat{y} / \partial z) \cdot (\partial z / \partial w)$$
$$\partial L / \partial w = (-0.4994) \cdot (0.1874) \cdot (2)$$
$$\partial L / \partial w \approx -0.1872$$

Аналогично для b:

$$\partial z / \partial b = 1 \quad (\text{потому что } z = x \cdot w + b, \text{ производная по } b \text{ равна } 1)$$

$$\partial L / \partial b = (\partial L / \partial \hat{y}) \cdot (\partial \hat{y} / \partial z) \cdot (\partial z / \partial b)$$
$$\partial L / \partial b = (-0.4994) \cdot (0.1874) \cdot (1)$$
$$\partial L / \partial b \approx -0.0936$$

Предварительный просмотр обратного прохода:

ОБРАТНЫЙ ПРОХОД (справа налево):

$$x=2 \longleftarrow z=1.1 \longleftarrow \hat{y}=0.7503 \longleftarrow L=0.0624$$

$\partial z / \partial w = 2$	$\partial \hat{y} / \partial z = 0.1874$	$\partial L / \partial \hat{y} = -0.4994$
└────────────────────────────────┘		
$\partial L / \partial w = -0.4994 \times 0.1874 \times 2 = -0.1872$		

ЭТАП 3: Обновление весов

Теперь используются найденные градиенты для коррекции весов.

Первое обновление (градиентный спуск):

правило обновления (градиентный спуск):

$$w_{\text{новый}} = w_{\text{старый}} - \alpha \cdot (\partial L / \partial w)$$
$$b_{\text{новый}} = b_{\text{старый}} - \alpha \cdot (\partial L / \partial b)$$

где α (альфа) — скорость обучения (скорость обучения) = 0,1

$$w_{\text{новый}} = 0.5 - 0.1 \cdot (-0.1872)$$
$$w_{\text{новый}} = 0.5 + 0.01872$$
$$w_{\text{новый}} = 0.5187$$

$$b_{\text{новый}} = 0.1 - 0.1 \cdot (-0.0936)$$
$$b_{\text{новый}} = 0.1 + 0.00936$$
$$b_{\text{новый}} = 0.1094$$

Градиент отрицательный → вес увеличивается (нужно двигаться в сторону, противоположную градиенту, чтобы уменьшить ошибку).

Проверяем: стало ли лучше?

Делаем ещё один прямой проход с новыми весами:

$$z = 2 \cdot 0.5187 + 0.1094 = 1.1468$$
$$\hat{y} = \sigma(1.1468) = 1 / (1 + e^{(-1.1468)}) \approx 0.7590$$
$$L = (1 - 0.7590)^2 \approx 0.0581$$

Было: $\hat{y} = 0.7503$, ошибка = 0.0624
Стало: $\hat{y} = 0.7590$, ошибка = 0.0581  Ошибка уменьшилась!

Предсказание стало ближе к 1, ошибка уменьшилась. **Работает!**

Повторяем этот процесс **тысячи раз** — и сеть обучается.

5. Пример более сложный: сеть с двумя слоями.

Вход Скрытый слой Выход
 $x \xrightarrow{w_1} [h] \xrightarrow{w_2} [\hat{y}] \rightarrow L$ (ошибка)

Здесь цепочка длиннее, но принцип тот же:

$$\partial L / \partial w_1 = (\partial L / \partial \hat{y}) \cdot (\partial \hat{y} / \partial h) \cdot (\partial h / \partial w_1)$$

↑	↑	↑
ошибка на	как выход	как скрытый
выходе	зависит	нейрон зависит
	от скрытого	от w_1
	нейрона	

Ошибка «течёт» от выхода ко входу , проходя через каждый слой — именно поэтому алгоритм называется **обратным распространением ошибок** .

ПРЯМОЙ ПРОХОД: $x \longrightarrow h \longrightarrow \hat{y} \longrightarrow L$

$\Longrightarrow$

ОБРАТНЫЙ ПРОХОД: $x \longleftarrow h \longleftarrow \hat{y} \longleftarrow L$

$\Longleftarrow$

(градиенты текут назад)

6. Итог: алгоритм по шагам

АЛГОРИТМ ОБРАТНОГО РАСПРОСТРАНЕНИЯ

Повторять много раз:

① ПРЯМОЙ ПРОХОД (Forward Pass)

- Подаём данные на вход
- Считаем выход каждого нейрона слой за слоем
- Получаем предсказание $\hat{y}$
- Считаем ошибку $L = (y - \hat{y})^2$

② ОБРАТНЫЙ ПРОХОД (Backward Pass)

- Считаем $\partial L / \partial \hat{y}$ на выходе
- По цепному правилу вычисляем градиенты для каждого веса, двигаясь от выхода ко входу

③ ОБНОВЛЕНИЕ ВЕСОВ

- $w = w - \alpha \cdot (\partial L / \partial w)$ для каждого веса
- Веса сдвигаются в сторону уменьшения ошибки

7. Частные вопросы

«Зачем именно ОБРАТНОЕ распространение? Почему нельзя сразу?»

Потому что цепное правило позволяет **переиспользовать вычисления**. Когда мы считаем градиент для последнего слоя, мы можем использовать этот результат для предпоследнего слоя и так далее. Это **экономит огромное количество вычислений**.

«Что если сеть глубокая (много слоёв)?»

Принцип тот же — просто цепочка длиннее:

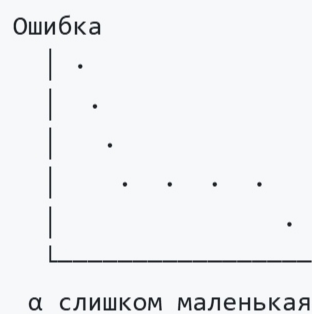
$$\partial L / \partial w_1 = (\partial L / \partial \hat{y}) \cdot (\partial \hat{y} / \partial h_3) \cdot (\partial h_3 / \partial h_2) \cdot (\partial h_2 / \partial h_1) \cdot (\partial h_1 / \partial w_1)$$

«Что такое скорость обучения α ?»

α слишком большая: перепрыгиваем минимум, сеть не учится

α слишком маленькая: учимся очень долго

α в самый раз: плавно спускаемся к минимуму ошибки



8. Аналогия-резюме

Нейросеть — это команда поваров, готовящих блюдо.

ПРЯМОЙ ПРОХОД:

Повар 1 (w_1) режет овощи → Повар 2 (w_2) жарит → Повар 3 (w_3) подаёт
Клиент пробует и говорит: "Пересолено!" (ошибка)

ОБРАТНЫЙ ПРОХОД:

Шеф-повар разбирается: кто виноват?

← Повар 3 слишком посолил при подаче? ($\partial L / \partial w_3$)

← Или Повар 2 добавил лишнюю соль при жарке? ($\partial L / \partial w_2$)

← Или Повар 1 взял солёные овощи? ($\partial L / \partial w_1$)

ОБНОВЛЕНИЕ ВЕСОВ:

Каждый повар корректирует свои действия пропорционально своей вине.

В следующий раз блюдо будет вкуснее! 🔍